TRAFFIX

INTELLIGENT TRAFFIC MANAGEMENT AND ROUTE OPTIMIZATION SYSTEM

COMPLETE FULL-STACK DEVELOPMENT MASTER PROMPT

PROJECT DOMAIN	Full Stack Web Development
PROBLEM STATEMENT	PS 01 – Intelligent Traffic Management and Route Optimization System
DELIVERABLE	Working frontend + backend + APIs + database + algorithms + prediction + simulations
CORE RULE	No decorative-only UI. Every key interaction must call working backend logic.

Prepared as a copy-ready specification for an AI coding agent or development team.

Important: use clearly labelled simulated data wherever live city traffic feeds are unavailable. Never present simulated conditions as live observations.

HOW TO USE THIS MASTER PROMPT

Copy the complete specification below into your coding agent. Instruct it to implement the application incrementally, run it, test it, and fix errors. It must create actual source files and a functioning server—not merely return a design or code snippets.

Project scope: This prompt combines every feature explicitly listed in the supplied PS 01 PDF with relevant capabilities found in existing navigation, community incident-reporting, adaptive-signal and traffic-simulation systems. These are implementation requirements, not claims of invention.

Mandatory items from the problem statement: route search and comparison; shortest/fastest routes; ETA and distance; incident reporting and map display; alternate routes; administrator review; 15–30 minute congestion prediction; duplicate detection and severity estimation; connected-road impact analysis; road/lane-blockage simulation; adaptive signals; emergency priority routing; and multi-factor route ranking.

1. PROJECT OBJECTIVE

Build a complete, responsive, technically credible full-stack web application named **TRAFFIX – Intelligent Traffic Management and Route Optimization System**. It should demonstrate how a smart traffic platform receives information, validates it, stores it, processes it, calculates decisions, and communicates results to users through APIs.

- Build an attractive and usable frontend, but make backend functionality the source of truth.
- Implement server-side business logic, database persistence, authentication, authorization, API validation and error handling.
- Connect maps, routes, incidents, predictions, signals, emergency vehicles and admin tools to real backend services.
- Use live external data only when a suitable API is configured. Otherwise use deterministic, documented simulation data.
- Do not fake functionality with buttons that only animate, frontend-only arrays, hardcoded success messages or static dashboard numbers.

2. REQUIRED TECHNOLOGY STACK

Layer	Recommended technology
Frontend	React + Vite + TypeScript; Tailwind CSS; Framer Motion; React Router
Maps	Leaflet + OpenStreetMap; geocoding/routing provider adapter
Charts	Recharts
API client	Axios; typed API service layer
Backend	Node.js + Express + TypeScript
Realtime	Socket.IO
Validation	Zod on the server; client validation for usability only
Database	PostgreSQL + PostGIS; Prisma ORM
Authentication	JWT access tokens; bcrypt password hashing; role-based authorization
Prediction service	Python + FastAPI + scikit-learn, or a documented TypeScript baseline if Python service is not feasible
Testing	Vitest/Jest; Supertest; Playwright or equivalent

Use modular provider adapters so the application can switch between a live routing/weather provider and a simulation provider without rewriting business logic.

3. APPLICATION ROLES AND ACCESS CONTROL

3.1 Public commuter

- Register, log in and manage profile.
- Search source and destination; compare shortest, fastest and recommended routes.
- View map layers for congestion, incidents, closures and predicted risk.
- Submit incident reports with location, type, severity and optional image.
- View report status and receive route/incident updates.
- Save favourite routes and view recent searches if implemented.

3.2 Traffic administrator

- Review, verify, reject and resolve incident reports.
- Inspect possible duplicates and incident evidence.
- View affected roads and network impact.
- Manage junctions, road closures and simulated traffic scenarios.
- Monitor congestion predictions, signal plans and emergency requests.
- View audit logs, usage analytics and operational reports.

3.3 Emergency vehicle operator

- Register or access an authorized ambulance/fire-service vehicle account.
- Submit an emergency trip with origin, destination and priority level.
- Share simulated or permission-based location updates.
- View priority route, affected junctions and estimated time saved.
- Track request states and completion history.

3.4 Traffic control operator

- Monitor configured intersections, vehicle density and queue length.
- Run adaptive signal optimization for selected junctions.
- Review recommended phase durations and before/after waiting-time estimates.
- Approve or simulate a signal plan; record who initiated the action.

Enforce role-based access on the backend. Hiding a button in the frontend is not authorization.

4. FRONTEND EXPERIENCE AND VISUAL DESIGN

Create a premium smart-city traffic operations interface—not a generic admin template. Use a dark navy/charcoal foundation, restrained electric-blue and cyan accents, clear typography, accessible contrast, subtle glass effects, responsive layouts and meaningful motion. Use green for clear flow, yellow for moderate congestion, orange for high congestion and red for severe congestion or verified critical incidents.

- Public landing page explaining the service and linking to navigation, incident reporting and sign-in.
- Navigation workspace with a large interactive map, source/destination controls, route alternatives and route explanation.

- Traffic operations dashboard with live/simulated status, key metrics, map and charts.
- Incident reporting form with validation, map pin selection, optional image upload and submission status.
- Incident detail view showing verification state, timestamp, severity and affected roads.
- Admin dashboard with filters, review queue, incident detail panel, audit history and actions.
- Emergency routing screen with route, ETA, junction priorities and trip status.
- Signal control screen with junction selector, density/queue metrics, proposed timing and simulation comparison.
- Responsive navigation, loading states, empty states, error states, confirmation dialogs and toast notifications.

All displayed operational data must come from API responses or an explicitly labelled seeded simulation dataset. Do not hardcode metrics in React components.

5. CORE FUNCTIONAL MODULES

MODULE 1 — SMART NAVIGATION AND MULTI-FACTOR ROUTE OPTIMIZATION

Implement source/destination search, coordinate selection and route alternatives on an interactive map.

- Accept typed place names and map-selected coordinates; geocode through a provider adapter.
- Calculate and display at least shortest-distance and fastest-time alternatives where route graph data permits.
- Calculate a recommended route using distance, estimated travel time, congestion, incidents, closures, fuel estimate and reliability.
- Show route geometry, distance, ETA, important turns/junctions, congestion and relevant incidents.
- Recalculate when an incident or closure affects the chosen route.
- Explain why a route is recommended using actual score components and data used.
- Handle no route, unavailable provider, invalid coordinates and timeout errors gracefully.

Algorithms: Implement Dijkstra for shortest path and A* for heuristic pathfinding on a graph. Represent roads as edges and intersections as nodes. Store edge length, speed estimate, capacity, current/predicted congestion, closure state and incident penalty. Ensure all algorithms run server-side.

Multi-factor scoring: Normalize component values before combining them. Make weights configurable and document them. A possible cost function is distance weight + time weight + congestion penalty + incident penalty + closure penalty + fuel estimate penalty – reliability benefit. Closed edges must be excluded, not merely given a small penalty.

MODULE 2 — INCIDENT REPORTING AND INTELLIGENT INCIDENT DETECTION

Allow users to report accidents, roadblocks, congestion, construction, lane closures, vehicle breakdowns, flooded roads and other obstructions.

- Capture incident type, description, coordinates, road/segment, severity, time, reporter and optional photograph.
- Validate coordinates, required fields, file type and file size on the server.
- Check nearby incidents within a configurable radius and time window.
- Compare incident type, distance, timestamps and text similarity to flag possible duplicates.
- Do not silently discard reports: link possible duplicates to a canonical incident or send them to admin review.
- Estimate severity using transparent rules or a trained model; show the reasons and confidence/limitations.
- Use statuses: Pending, Under Review, Verified, Rejected, Resolved.
- Only authorized operators/admins can verify, reject or resolve reports.
- Broadcast verified changes to affected connected clients using Socket.IO.

Incident detection must be auditable: retain the original report, duplicate match, reviewer action and timestamps.

MODULE 3 — AI-BASED CONGESTION PREDICTION (15–30 MINUTES)

Build a prediction pipeline using historical or simulated traffic observations. Predict congestion 15 and 30 minutes ahead; do not label simulated predictions as live city forecasts.

- Features: time of day, day of week, road capacity, observed/simulated vehicle count, average speed, historical patterns, weather when available, active incidents and closures.
- Produce predicted density/flow, average speed estimate, congestion class and risk score for each monitored road segment.
- Use interpretable baseline models first, such as regression, random forest or gradient boosting, depending on available data.
- Train/test split by time where possible to avoid leakage; report MAE/RMSE or appropriate classification metrics.
- Display current versus predicted traffic and 15/30-minute risk on a colour-coded map.
- Store prediction timestamp, horizon, model version, input provenance and prediction output.
- Compare predictions with later observations when available and calculate model performance.
- If there is insufficient training data, use a transparent rules-based baseline and label it as such.

MODULE 4 — ROAD NETWORK IMPACT ANALYSIS AND BLOCKAGE SIMULATION

- Model the road network as a graph of junction nodes and road-segment edges.
- When an incident is verified, identify directly affected edges and connected alternative paths.
- Estimate downstream impact using graph traversal, edge capacity, connectivity and detour cost.
- Simulate full road closure, one-lane blockage or reduced road capacity.
- Recalculate routes after applying the simulated change.
- Show affected road segments, likely diversion paths and estimated added travel time.
- Allow admin/operator to start, inspect and end a simulation without changing the real incident record.
- Persist simulation scenarios and results for repeatable demonstrations.

MODULE 5 — ADAPTIVE TRAFFIC SIGNAL OPTIMIZATION

- Create configurable junctions with approaches, lanes, signal phases, minimum/maximum green time and cycle constraints.
- Accept vehicle density and queue-length observations from seeded simulation or a configured data source.
- Calculate adaptive phase durations based on queue length, demand and fairness constraints.
- Simulate traffic before and after optimization using the same initial conditions.
- Compare average waiting time, queue length, throughput and delay.
- Show current timing, proposed timing, reason for adjustment and simulation outcome.
- Prevent unsafe timing values with server-side bounds and conflict checks.
- Record operator approval and every simulated or applied change.

For a hackathon prototype, signal changes may be simulated. Clearly label the interface as a simulation and do not claim control of real roadside hardware.

MODULE 6 — EMERGENCY VEHICLE PRIORITY ROUTING

- Support ambulance and fire-service emergency trips.
- Calculate routes using distance, predicted congestion, verified incidents and closures.
- Rank candidate routes and show estimated arrival time.

- Simulate priority passage through configured junctions and propose green-corridor timing.
- Show junctions on the emergency route and the priority action planned at each one.
- Compare emergency-route ETA with a normal-route baseline and calculate estimated time saved.
- Update emergency trip state and notify authorized traffic operators through realtime events.
- Handle route blockage, failed location update, cancellation and trip completion.
- Keep a log of the route, data used, priority actions and estimated versus actual/simulated arrival.

Do not claim that the prototype can command real traffic signals or dispatch real emergency vehicles unless authorized hardware and operational integrations actually exist.

MODULE 7 — REAL-TIME TRAFFIC AND USER UPDATES

- Use Socket.IO rooms or channels for relevant route, incident, junction and admin updates.
- Broadcast verified incident changes, road closure changes, prediction refreshes, signal simulation updates and emergency trip status.
- Authenticate socket connections and authorize room subscriptions.
- Avoid broadcasting private user information.
- Reconnect safely and fetch current state after reconnection so missed events do not leave stale UI.

MODULE 8 — ADMINISTRATION, AUDIT AND ANALYTICS

- Incident queue with filters by status, type, severity, date and area.
- Incident review page with map, reports, possible duplicates and decision controls.
- Traffic overview showing segment congestion, top affected roads and active closures.
- Prediction analytics showing forecast horizon, model version and evaluation metrics.
- Signal optimization history with before/after comparisons.
- Emergency trip history and estimated time saved.
- Export operational summaries as CSV where appropriate.
- Audit log for sensitive actions: actor, action, target, timestamp and relevant before/after values.

6. BACKEND ARCHITECTURE AND API CONTRACT

Organize the backend into routes/controllers, services, repositories/data access, validation schemas, middleware, realtime handlers and provider adapters. Keep route handlers thin; business logic belongs in services.

6.1 Suggested API endpoints

Method + endpoint	Purpose / access
POST /api/auth/register	Create account; public
POST /api/auth/login	Authenticate; public
GET /api/auth/me	Current user; authenticated
GET /api/routes/alternatives	Calculate route alternatives; authenticated or public policy
POST /api/incidents	Submit incident; authenticated
GET /api/incidents	List/filter incidents; role/area scoped
GET /api/incidents/:id	Incident detail; authorized
PATCH /api/incidents/:id/status	Verify/reject/resolve; admin
POST /api/incidents/:id/impact-analysis	Run network impact analysis; admin/operator
POST /api/simulations/closures	Create closure/lane-blockage scenario; admin/operator
GET /api/simulations/:id	Read simulation results; authorized
POST /api/predictions/run	Run 15/30-minute prediction; admin/operator or scheduled job
GET /api/predictions	Fetch predictions by area/segment/time
GET /api/junctions	List monitored junctions; authorized
POST /api/signals/optimize	Calculate adaptive timing plan; operator/admin
POST /api/signals/simulate	Run before/after signal simulation; operator/admin
POST /api/emergency/trips	Create emergency trip; emergency role
PATCH /api/emergency/trips/:id/status	Update trip status; authorized
POST /api/emergency/trips/:id/priority-route	Calculate priority route; emergency/operator
GET /api/analytics/overview	Dashboard metrics; admin/operator
GET /api/health	Service health check; public or restricted

The exact endpoint design may be refined, but every feature must have a documented API contract, request schema, response schema, authorization rule and error behavior.

6.2 API response conventions

```
{
"success": true,
"data": { },
"message": "Request completed",
"meta": {
"requestId": "server-generated-id",
"timestamp": "ISO-8601"
}
}
```

Use appropriate HTTP status codes: 200/201 for success, 400 for invalid input, 401 unauthenticated, 403 forbidden, 404 missing resource, 409 conflict, 422 semantic validation failure and 500 unexpected server error. Never return stack traces or secrets to the browser.

6.3 Example incident API contract

```
POST /api/incidents
Content-Type: application/json
Authorization: Bearer <access-token>

{
"type": "ACCIDENT",
"description": "Collision near the junction",
"latitude": 15.85,
"longitude": 74.50,
"severity": "HIGH",
"roadSegmentId": "segment-id"
}

Example response:
{
"success": true,
"data": {
"incidentId": "generated-id",
"status": "PENDING",
"possibleDuplicate": false
},
"message": "Incident submitted for review"
}
```

7. DATABASE DESIGN

Create normalized relational models with proper foreign keys, indexes, timestamps and migrations. Use PostGIS geography/geometry columns and spatial indexes for location queries.

- User: id, name, email, passwordHash, role, createdAt, updatedAt, status.
- Incident: id, reporterId, type, description, location, roadSegmentId, severity, status, createdAt, updatedAt, resolvedAt.
- IncidentReport/Match: submitted report, canonical incident, duplicate score/reason and review outcome.
- RoadNode: id, externalReference, name, location, junction metadata.
- RoadSegment: id, startNodeId, endNodeId, geometry, length, capacity, speedLimit, closure status.
- TrafficObservation: segmentId, observedAt, vehicleCount, averageSpeed, queueLength, source and simulated flag.
- TrafficPrediction: segmentId, predictedAt, horizonMinutes, congestionScore, predictedSpeed, modelVersion, dataSource.
- Junction: id, name, location, configuration and active status.
- SignalPhase/SignalPlan: junction, phase sequence, timing constraints, proposed timings, approval and simulation results.
- EmergencyVehicle: id, operatorId, type, registration label and availability.
- EmergencyTrip: vehicleId, origin, destination, route, status, ETA, priority plan and timestamps.
- SimulationScenario: type, parameters, createdBy, status and results.
- AuditLog: actorId, action, targetType, targetId, before/after summary and createdAt.
- SavedRoute / RouteHistory: user, origin, destination, route summary and timestamps if included.

Use migrations and seed scripts. Include realistic, clearly labelled sample users, road graph, junctions, traffic observations and incidents for a repeatable demo. Never seed real people's personal information.

8. VALIDATION, SECURITY AND RELIABILITY

- Validate every request body, query parameter and path parameter on the backend using Zod or equivalent.
- Normalize email addresses; enforce unique accounts; validate password strength.

- Hash passwords using bcrypt or an equivalent secure password-hashing algorithm.
- Use short-lived access tokens and a considered refresh-token strategy; store tokens securely.
- Enforce role and resource-level authorization on every protected endpoint.
- Use rate limits for login, registration, incident submission and expensive prediction/route operations.
- Validate geographic bounds, numeric ranges, enum values, timestamps and uploaded files.
- Use parameterized database access through Prisma; avoid raw SQL injection risks.
- Configure CORS narrowly; use Helmet; avoid logging credentials or tokens.
- Add request IDs, structured logs, centralized error handling and health checks.
- Use database transactions for multi-step writes and prevent duplicate state transitions.
- Handle external provider timeouts, rate limits, unavailable services and malformed responses.
- Keep API keys and database credentials in server-side environment variables.

9. REAL-TIME EVENT DESIGN

- incident:created — new report submitted; visible to authorized review workflow.
- incident:verified — verified incident and map state updated.
- incident:resolved — incident removed from active hazard layer.
- traffic:prediction-updated — new forecast available for subscribed area/segments.
- road:status-changed — closure or capacity change applied.
- signal:plan-updated — signal simulation or approved plan changed.
- emergency:trip-updated — trip status/ETA/priority route changed.

Document each event payload, room authorization and reconnect behavior. The frontend must update from the event or refetch authoritative state from the API.

10. SIMULATION AND DATA INTEGRITY

The problem statement permits simulated traffic, weather and incident data when clearly identified. Build a deterministic simulation mode that makes the system demonstrable without paid commercial feeds.

- Create a small sample road graph with several intersections and alternative paths.
- Seed baseline traffic profiles by hour/day and different road capacities.
- Allow simulation controls to increase vehicle density, add an accident, close a lane or change weather input.
- Use a seeded random generator or fixed scenario fixtures so repeated demos are reproducible.
- Display a visible SIMULATED DATA label anywhere simulated conditions or forecasts appear.
- Keep data provenance fields so live and simulated observations cannot be confused.
- If a live provider is configured, show provider/source and last-updated time.
- Do not fabricate live traffic, live GPS, actual signal control or emergency dispatch.

11. FRONTEND-BACKEND INTEGRATION RULES

1. Create a central typed API client with base URL from environment configuration.
2. Create separate API modules for auth, routes, incidents, predictions, signals, emergency trips and analytics.
3. All data-driven screens must load from backend endpoints.

4. Forms must submit to real endpoints, display server validation errors and show success only after a successful response.
5. Buttons must call a real API or perform a clearly local UI action; do not leave inert controls.
6. Use loading, disabled, success, empty and failure states.
7. Use optimistic updates only when rollback behavior is implemented.
8. Use Socket.IO for supported live updates and refetch after reconnect.
9. Never place database credentials, private API keys or privileged logic in frontend code.
10. Add integration tests proving that frontend actions cause backend state changes.

12. RECOMMENDED REPOSITORY STRUCTURE

```
traffix/
client/
src/
app/
components/
pages/
features/
auth/
navigation/
incidents/
predictions/
signals/
emergency/
admin/
services/
hooks/
types/
styles/
server/
src/
config/
middleware/
routes/
controllers/
services/
repositories/
validators/
algorithms/
realtime/
providers/
utils/
app.ts
server.ts
prisma/
schema.prisma
migrations/
seed.ts
prediction-service/
app/
main.py
models/
schemas/
services/
requirements.txt
shared/
types/
docs/
architecture.md
api-contract.md
demo-scenarios.md
.env.example
docker-compose.yml
README.md
```

13. REQUIRED DEMONSTRATION SCENARIOS

Build these end-to-end scenarios so a judge can inspect the actual system behavior and backend communication.

Scenario A — Commuter route search

1. User enters a source and destination.
2. Frontend calls the route alternatives API.
3. Backend obtains road graph/provider routes and computes shortest, fastest and recommended choices.
4. Frontend displays route geometry, ETA, distance and score explanation.
5. User selects an alternative and the selected route is retained in the UI.

Scenario B — Accident report changes route

1. User submits an accident report through the incident form.
2. Backend validates and stores the report as Pending.
3. Duplicate detection checks nearby recent incidents.
4. Admin reviews and verifies the report.
5. Backend calculates affected road segments and updates the active network state.
6. Route service recalculates alternatives; connected clients receive a realtime event.
7. The commuter map displays the incident and revised route.

Scenario C — Congestion prediction

1. Operator opens a road segment prediction view.
2. Backend loads historical/simulated observations and relevant features.
3. Prediction service calculates 15-minute and 30-minute forecasts.
4. Backend stores results with model version and data provenance.
5. Frontend displays risk map, forecast chart and evaluation metrics.

Scenario D — Road blockage impact

1. Admin selects a road segment and simulates a lane blockage or full closure.
2. Backend creates a scenario without overwriting the real road record.
3. Graph algorithm calculates affected connectivity and detours.
4. UI shows affected segments, alternative paths and estimated delay.
5. Admin ends the scenario and the normal network state is restored.

Scenario E — Adaptive traffic signal

1. Operator selects a junction with queue and density observations.
2. Backend computes a proposed signal timing plan within configured safety bounds.
3. Simulation runs before/after using comparable initial conditions.
4. Dashboard compares waiting time, queue length and throughput.
5. Operator records approval or keeps the result as a simulation only.

Scenario F — Emergency priority route

1. Authorized emergency operator submits an ambulance/fire-service trip.

2. Backend calculates priority route using distance, congestion and incidents.
3. Backend identifies junctions on the route and generates simulated priority actions.
4. Dashboard compares normal and priority ETA and estimated time saved.
5. Trip status and route changes are broadcast to authorized operators.

14. TESTING AND ACCEPTANCE CRITERIA

Do not declare the project complete until the following checks pass.

- A new user can register, log in and retrieve their own profile.
- A normal user cannot access admin-only APIs.
- Invalid incident forms are rejected by the server with useful validation messages.
- A valid incident is persisted and can be retrieved after page refresh/server restart.
- Nearby duplicate reports are flagged and visible for review.
- An admin can verify and resolve an incident; unauthorized users cannot.
- A route search calls the backend and returns alternatives from the routing engine or labelled simulation graph.
- Shortest route and fastest route can differ in a test graph where distance and speed differ.
- A verified closure changes the route result when an alternative exists.
- 15/30-minute prediction results are generated by the prediction pipeline and include provenance.
- Signal optimization returns bounded timings and a before/after simulation comparison.
- Emergency routing uses priority scoring and returns junction actions and ETA comparison.
- Realtime clients receive authorized updates and recover state after reconnect.
- The application handles missing provider credentials and unavailable external services gracefully.
- No primary button, form or filter is decorative or inert.

15. BUILD ORDER — IMPLEMENT IN PHASES

1. **Phase 1: Foundation.** Create monorepo, React app, Express server, database, Prisma schema, migrations, seed data, environment setup and health check.
2. **Phase 2: Authentication.** Implement registration, login, password hashing, JWT, role authorization and protected routes.
3. **Phase 3: Core navigation.** Build map UI, geocoding/provider adapter, road graph, Dijkstra/A*, route alternatives and multi-factor scoring.
4. **Phase 4: Incidents.** Implement reporting, validation, image handling, duplicate detection, severity, admin review and incident map layer.
5. **Phase 5: Network impact.** Implement affected-road analysis, closure/lane-blockage scenarios and route recalculation.
6. **Phase 6: Prediction.** Build data pipeline, baseline model, 15/30-minute outputs, persistence, map layer and evaluation.
7. **Phase 7: Signals.** Build junction configuration, adaptive timing logic, simulation and comparison dashboard.
8. **Phase 8: Emergency.** Build emergency accounts/trips, priority routing, junction plan simulation and status updates.
9. **Phase 9: Realtime and analytics.** Add Socket.IO, admin analytics, audit log, exports and reconnect recovery.
10. **Phase 10: Quality.** Run unit, API, integration and end-to-end tests; fix errors; document setup and demo scenarios.

16. ENVIRONMENT AND RUN INSTRUCTIONS

Create a complete .env.example with placeholders only. Include database URL, JWT secret, client origin, server port, map/routing provider configuration and prediction-service URL. Never commit real secrets.

```
# Example commands — adapt scripts to the generated project
npm install
docker compose up -d db
npm run db:migrate
npm run db:seed
npm run dev

# If the prediction service is separate:
cd prediction-service
python -m venv .venv
# activate the virtual environment
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8001
```

Provide a README with prerequisites, setup, migrations, seed accounts (development only), environment variables, how to run each service, API documentation, test commands and known limitations.

17. FINAL INSTRUCTIONS TO THE CODING AGENT

Follow these instructions strictly:

1. Do not stop after creating the frontend. Build the database, backend, APIs and integration.
2. Do not create fake buttons, fake API responses, hardcoded dashboard statistics or frontend-only incident submissions.
3. Do not claim that an API, model, database or integration works unless it has actually been implemented and tested.
4. Do not silently omit any mandatory feature. If a feature requires simulation, implement the simulation and label it clearly.
5. Create real source files in the project directory and keep the code organized and maintainable.
6. After each phase, run the app and relevant tests; fix errors before proceeding.
7. Use database persistence for all important records.
8. Document the request/response flow for every major feature so a student can explain how the frontend, backend, API and database communicate.
9. When an external service is unavailable, use a clearly identified fallback provider rather than returning fabricated live data.
10. At completion, provide a feature checklist, architecture explanation, API list, database model summary, run commands, test results and limitations.

FINAL DELIVERABLE: A fully connected, working full-stack intelligent traffic management application that demonstrates all PS 01 mandatory features, plus the relevant capabilities of established navigation, incident-reporting, adaptive-signal and traffic-simulation systems. The application must be demonstrable end-to-end, with real server-side processing and persistent database integration.

REFERENCE TO THE ORIGINAL PROBLEM STATEMENT

The supplied PS 01 document specifies route search, route comparison, shortest/fastest routes, ETA and distance, incident reporting, incident map display, alternate routes, admin review, AI congestion prediction 15–30 minutes ahead, duplicate incident detection, severity estimation, connected-road impact analysis, road/lane blockage simulation, adaptive signal optimization, emergency vehicle priority routing and multi-factor route ranking. It recommends React/Angular/Vue, Node.js/Django/Spring Boot, MySQL/PostgreSQL/MongoDB, OpenStreetMap/Leaflet, Dijkstra/A*, and historical or simulated data. It explicitly states that live commercial map or traffic API integration is not mandatory if simulated data is clearly identified.

This master prompt expands those requirements into a practical full-stack implementation specification. Choose feasible integrations, disclose simulation boundaries, and prioritize a working end-to-end system over superficial feature claims.